ATME Dienstleistung Management System

Database Design & Documentation
Entity-Relationship Models (ERD & ERC), Schema Reference, Design Rationale

Project	ATME Dienstleistung - Company Management Platform
Backend	Django 6 + Django REST Framework + Django Channels
Database	PostgreSQL
Document	Database Schema & ER Models
Version	1.0
Status	Internal / Pre-launch

Contents

1. System Overview

2. Technology Stack

3. Conceptual ER Diagram (ERD)

4. Relational Schema Diagram (ERC)

5. Detailed Table Reference

6. Relationships & Referential Integrity

7. Design Decisions & Engineering Practices

8. Roadmap

1. System Overview

ATME Dienstleistung is a company management platform for a services business that delivers **software/IT projects** as well as **facility services** (cleaning and security). The backend is a modular Django REST API organised into five applications:

accounts	Custom user model for employees and administrators, with roles, seniority and profile data. Provides JWT authentication.
customers	Clients that order services (companies, schools, government, etc.) and the list of services each one buys.
projects	Software/IT delivery: projects, ordered tasks, team assignment and discussion comments.
operations	Routine facility work: physical sites and the assignment of cleaning/security employees to them.
chat	Real-time messaging and incident documentation (group chats, manager-employee direct chats, image attachments).

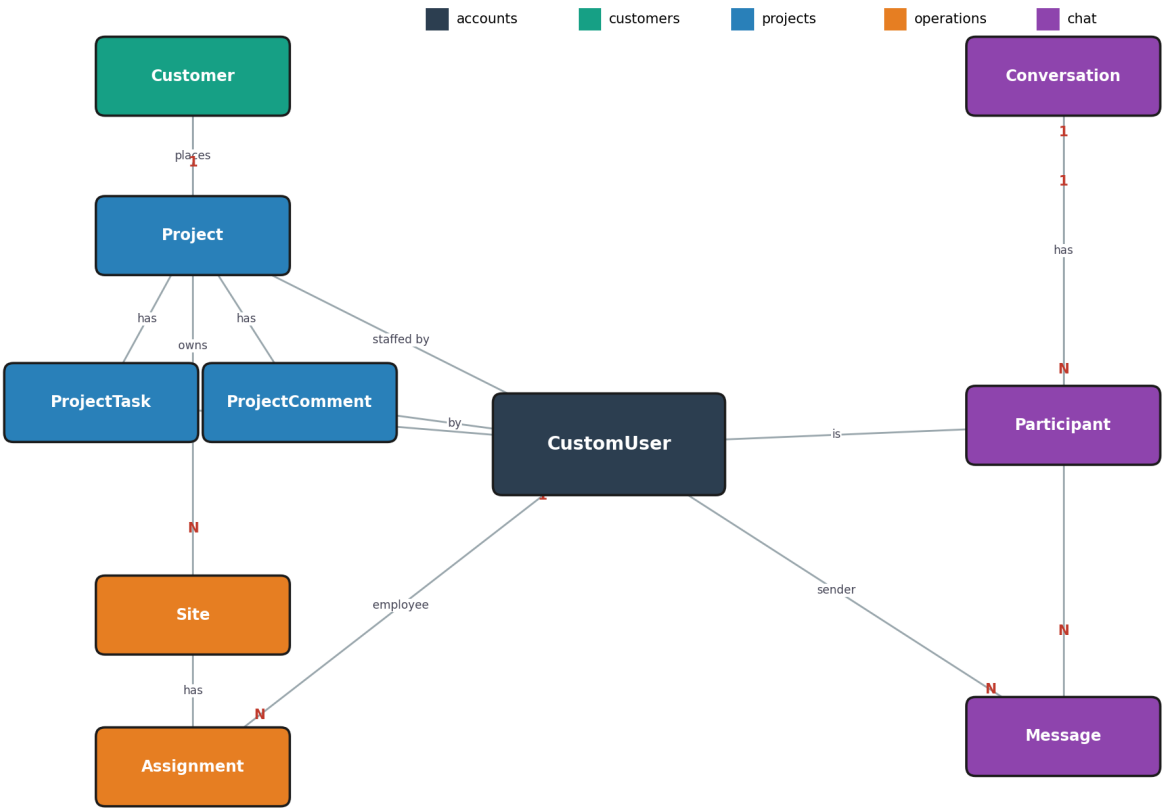
The data model is fully normalised, uses the database's referential integrity (foreign keys with explicit delete policies), enumerated value sets for controlled vocabularies, and a single central user table referenced by every operational entity.

2. Technology Stack

Layer	Technology	Purpose
Language	Python 3.12	Application runtime.
Framework	Django 6 + Django REST Framework	ORM, migrations, REST API.
Realtime	Django Channels + Daphne (ASGI)	WebSocket chat delivery.
Database	PostgreSQL	Relational store; ArrayField support.
Auth	SimpleJWT (access/refresh)	Stateless token authentication.
Cache/Broker	Redis + Celery	Channel layer (prod) & async tasks.
Tooling	django-filter, drf-spectacular, django-environ	Filtering, OpenAPI schema, config.

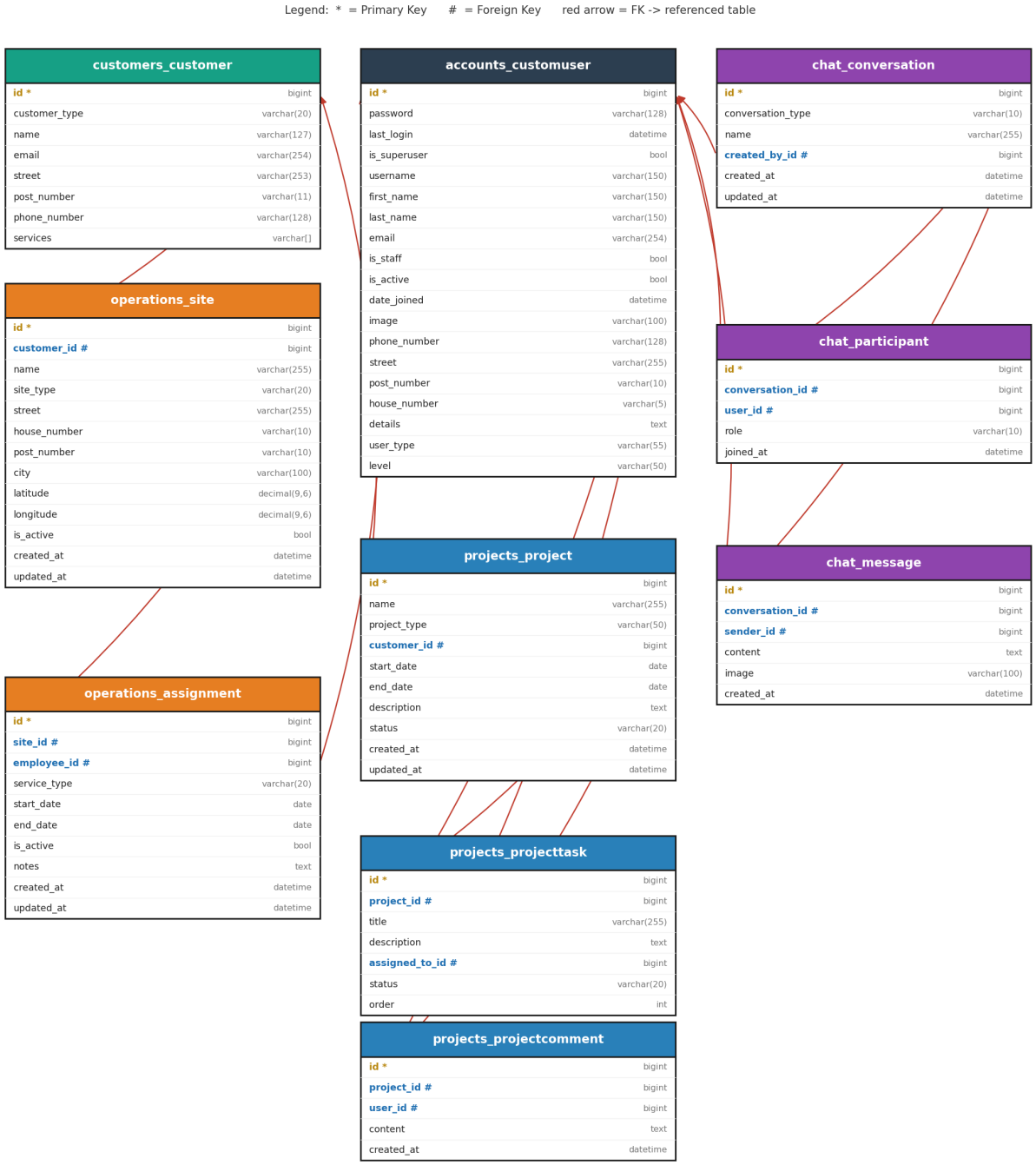
3. Conceptual ER Diagram (ERD)

The conceptual model shows the system's entities and the relationships between them with their cardinalities (1, N, M). **CustomUser** is the central entity referenced across all modules.



4. Relational Schema Diagram (ERC)

The relational model shows every table with its columns, primary keys (*), foreign keys (#) and the foreign-key references that enforce referential integrity.



5. Detailed Table Reference

accounts_customuser (CustomUser)

System users (employees and administrators). Extends Django's AbstractUser and is the project's AUTH_USER_MODEL.

Column	Type	Key / Constraints	Description
id	BIGINT	PK, auto, not null	Primary key.
password	VARCHAR(128)	not null	Salted PBKDF2 password hash.
last_login	TIMESTAMP	null	Last successful login.
is_superuser	BOOLEAN	default false	Full unrestricted access.
username	VARCHAR(150)	unique, not null	Login identifier.
first_name	VARCHAR(150)		Given name.
last_name	VARCHAR(150)		Family name.
email	VARCHAR(254)		Email address (not unique).
is_staff	BOOLEAN	default false	Manager / admin-site access.
is_active	BOOLEAN	default true	Account enabled flag.
date_joined	TIMESTAMP	default now()	Registration timestamp.
image	VARCHAR(100)	null	Profile picture path (media/profile_pics).
phone_number	VARCHAR(128)	unique, null	Phone in E.164 format (region DE).
street	VARCHAR(255)	not null	Street name.
post_number	VARCHAR(10)	not null	Postal code.
house_number	VARCHAR(5)	not null	House number.
details	TEXT	null	Free-form notes.
user_type	VARCHAR(55)	default 'SECURITY'	Role/specialty enum (see choices).
level	VARCHAR(50)	default 'MID'	Seniority enum (JUNIOR/MID/SENIOR).

Enumerations: user_type: SECURITY, CLEANER, FRONTEND_DEV, BACKEND_DEV, AI_DEV, FULLSTACK_DEV, DATA_ENTRY | level: JUNIOR, MID, SENIOR
Notes: UNIQUE(username), UNIQUE(phone_number). Also carries Django M2M to auth.Group and auth.Permission.

customers_customer (Customer)

Clients that order services (software/IT or facility services such as cleaning and security).

Column	Type	Key / Constraints	Description
id	BIGINT	PK, auto, not null	Primary key.
customer_type	VARCHAR(20)	default 'COMPANY'	Client category enum.
name	VARCHAR(127)	not null	Client name.
email	VARCHAR(254)	not null	Contact email.
street	VARCHAR(253)	not null	Street name.
post_number	VARCHAR(11)	null	Postal code.
phone_number	VARCHAR(128)	not null	Phone E.164 (region DE).
services	VARCHAR[]	null	PostgreSQL array of ordered services (e.g. Cleaning, Security, Web App).

Enumerations: customer_type: COMPANY, SCHOOL, UNIVERSITY, GOVERNMENT, CHILDREN, HOMELESS, ORGANIZATION
Notes: Uses PostgreSQL ArrayField (django.contrib.postgres) for the services list.

projects_project (Project)

A software/IT project delivered for a customer.

Column	Type	Key / Constraints	Description
id	BIGINT	PK, auto, not null	Primary key.
name	VARCHAR(255)	not null	Project name.
project_type	VARCHAR(50)	default 'WEB_APP'	Project category enum.
customer_id	BIGINT	FK, -> customers_customer, CASCADE, not null	Owning client.
start_date	DATE	not null	Planned start.
end_date	DATE	not null	Planned end.
description	TEXT	not null	Scope/description.
status	VARCHAR(20)	default 'PENDING'	Lifecycle status enum.
created_at	TIMESTAMP	auto_now_add	Row creation time.
updated_at	TIMESTAMP	auto_now	Last update time.

Enumerations: project_type: WEB_APP, MOBILE_APP, UI_UX, API_DEV, OTHER | status: PENDING, STARTED, ON_WORK, FINISHED
Notes: M2M employees -> accounts_customuser via join table projects_project_employees.

projects_projecttask (ProjectTask)

An ordered step/task inside a project plan.

Column	Type	Key / Constraints	Description
id	BIGINT	PK, auto, not null	Primary key.
project_id	BIGINT	FK, -> projects_project, CASCADE, not null	Parent project.
title	VARCHAR(255)	not null	Task title.
description	TEXT	null	Task details.
assigned_to_id	BIGINT	FK, -> accounts_customuser, SET_NULL, null	Responsible employee.
status	VARCHAR(20)	default 'ON_SCHADUEL'	Task status enum.
order	INTEGER	default 1	Step order (positive).

Enumerations: status: ON_SCHADUEL, IN_PROGRESS, DONE
Notes: assigned_to uses SET_NULL so deleting an employee keeps task history.

projects_projectcomment (ProjectComment)

A discussion message attached to a project.

Column	Type	Key / Constraints	Description
id	BIGINT	PK, auto, not null	Primary key.
project_id	BIGINT	FK, -> projects_project, CASCADE, not null	Parent project.
user_id	BIGINT	FK, -> accounts_customuser, CASCADE, not null	Author.
content	TEXT	not null	Comment body.
created_at	TIMESTAMP	auto_now_add	Posted time.

Notes: Authors can delete their own comment only within 5 minutes (enforced in the API).

operations_site (Site)

A physical location (building, school, company...) where routine cleaning/security work is carried out.

Column	Type	Key / Constraints	Description
id	BIGINT	PK, auto, not null	Primary key.

Column	Type	Key / Constraints	Description
customer_id	BIGINT	FK, -> customers_customer, SET_NULL, null	Owning client (optional).
name	VARCHAR(255)	not null	Site name.
site_type	VARCHAR(20)	default 'BUILDING'	Location category enum.
street	VARCHAR(255)	not null	Street name.
house_number	VARCHAR(10)		House number.
post_number	VARCHAR(10)		Postal code.
city	VARCHAR(100)		City.
latitude	DECIMAL(9,6)	null	Geo latitude (for future check-in).
longitude	DECIMAL(9,6)	null	Geo longitude (for future check-in).
is_active	BOOLEAN	default true	Active flag.
created_at	TIMESTAMP	auto_now_add	Row creation time.
updated_at	TIMESTAMP	auto_now	Last update time.

Enumerations: site_type: BUILDING, COMPANY, SCHOOL, UNIVERSITY, GOVERNMENT, HOSPITAL, OTHER
Notes: customer is nullable (SET_NULL): sites may be standalone or tied to a client.

operations_assignment (Assignment)

Links an employee (cleaner/security) to a site for a given service.

Column	Type	Key / Constraints	Description
id	BIGINT	PK, auto, not null	Primary key.
site_id	BIGINT	FK, -> operations_site, CASCADE, not null	Target site.
employee_id	BIGINT	FK, -> accounts_customuser, CASCADE, not null	Assigned employee.
service_type	VARCHAR(20)	not null	Service enum (CLEANING/SECURITY).
start_date	DATE	null	Assignment start.
end_date	DATE	null	Assignment end.
is_active	BOOLEAN	default true	Active flag.
notes	TEXT	null	Free notes.
created_at	TIMESTAMP	auto_now_add	Row creation time.
updated_at	TIMESTAMP	auto_now	Last update time.

Enumerations: service_type: CLEANING, SECURITY
Notes: API rule: employee.user_type must match service_type (CLEANER<->CLEANING, SECURITY<->SECURITY).

chat_conversation (Conversation)

A chat container: a group or a direct (manager<->employee) conversation.

Column	Type	Key / Constraints	Description
id	BIGINT	PK, auto, not null	Primary key.
conversation_type	VARCHAR(10)	default 'GROUP'	GROUP or DIRECT.
name	VARCHAR(255)	blank	Group name (empty for direct).
created_by_id	BIGINT	FK, -> accounts_customuser, SET_NULL, null	Creator.
created_at	TIMESTAMP	auto_now_add	Created time.
updated_at	TIMESTAMP	auto_now	Last activity time.

Enumerations: conversation_type: GROUP, DIRECT
Notes: M2M participants -> accounts_customuser through chat_participant.

chat_participant (Participant)

Membership of a user in a conversation (through model).

Column	Type	Key / Constraints	Description
id	BIGINT	PK, auto, not null	Primary key.
conversation_id	BIGINT	FK, -> chat_conversation, CASCADE, not null	Conversation.
user_id	BIGINT	FK, -> accounts_customuser, CASCADE, not null	Member.
role	VARCHAR(10)	default 'MEMBER'	MEMBER or ADMIN.
joined_at	TIMESTAMP	auto_now_add	Join time.

Enumerations: role: MEMBER, ADMIN
Notes: UNIQUE(conversation_id, user_id) - a user joins a conversation once.

chat_message (Message)

A single chat message: text and/or an image (for documenting situations).

Column	Type	Key / Constraints	Description
id	BIGINT	PK, auto, not null	Primary key.
conversation_id	BIGINT	FK, -> chat_conversation, CASCADE, not null	Owning conversation.
sender_id	BIGINT	FK, -> accounts_customuser, SET_NULL, null	Author.
content	TEXT	blank	Message text.
image	VARCHAR(100)	null	Attached image path (media/chat/).
created_at	TIMESTAMP	auto_now_add	Sent time.

Notes: API rule: a message must contain text or an image.

6. Relationships & Referential Integrity

Parent (1)	Child (N)	Foreign key	Cardinality	On delete
customers_customer	projects_project	customer_id	1 : N	CASCADE
customers_customer	operations_site	customer_id	1 : N	SET_NULL
projects_project	projects_projecttask	project_id	1 : N	CASCADE
projects_project	projects_projectcomment	project_id	1 : N	CASCADE
projects_project	accounts_customuser	employees (M2M)	N : M	join table
accounts_customuser	projects_projecttask	assigned_to_id	1 : N	SET_NULL
accounts_customuser	projects_projectcomment	user_id	1 : N	CASCADE
operations_site	operations_assignment	site_id	1 : N	CASCADE
accounts_customuser	operations_assignment	employee_id	1 : N	CASCADE
chat_conversation	chat_participant	conversation_id	1 : N	CASCADE
chat_conversation	chat_message	conversation_id	1 : N	CASCADE
accounts_customuser	chat_participant	user_id	1 : N	CASCADE
accounts_customuser	chat_message	sender_id	1 : N	SET_NULL
accounts_customuser	chat_conversation	created_by_id	1 : N	SET_NULL

Delete policies. CASCADE is used where child rows have no meaning without the parent (e.g. a project's tasks and comments, a conversation's messages). SET_NULL is used to preserve historical records when a referenced user is removed (e.g. message sender, task assignee, site owner).

7. Design Decisions & Engineering Practices

• **Custom user model.** A single CustomUser (AbstractUser) is the AUTH_USER_MODEL, extended with domain roles (user_type), seniority (level) and profile/address fields - avoiding a separate profile table and keeping authentication first-class.

• **Stateless JWT auth.** Authentication uses short-lived access tokens and refresh tokens (SimpleJWT), so the API scales horizontally without server-side sessions. The same tokens authenticate WebSocket connections via a custom Channels middleware.

• **Role-based authorization.** Dedicated DRF permission classes enforce business rules: data-entry vs. superuser for master data, senior full-stack for project management, project-membership checks, and manager (staff/superuser) rules for chat.

• **Normalisation & integrity.** Every relationship is a real foreign key with an explicit on_delete policy. Controlled vocabularies are modelled as enumerated choices, and uniqueness is enforced at the database level (username, phone, one membership per conversation).

• **Shared Customer entity.** Customers are reused across both business lines (software and facility services). An operations Site links to a Customer optionally (nullable), allowing both client-owned and standalone locations.

• **Domain validation.** Application-level invariants protect data quality - e.g. an Assignment requires the employee's specialty to match the service type, and a chat Message must carry text or an image.

• **Real-time architecture.** Chat is built REST-first (persisted, paginated, image uploads) then layered with Django Channels for live delivery. Messages sent over REST are also broadcast to WebSocket subscribers through the channel layer, so both paths stay consistent.

• **PostgreSQL features.** The services catalogue uses a native PostgreSQL array (ArrayField) instead of a join table where a lightweight, queryable list is sufficient.

• **Configuration & secrets.** Settings are environment-driven (django-environ); secrets are kept out of source. Redis/Celery are wired for asynchronous work and a production-grade channel layer.

• **Versioned schema.** All schema changes are tracked as Django migrations, giving a reproducible, auditable history of the database structure.

8. Roadmap

• Operations: shift scheduling (days/times) and geo-verified attendance (check-in/out) using the site coordinates.

• Multi-tenancy: an Organization boundary to evolve the platform into a per-company SaaS subscription.

• Containerisation: Docker Compose for web, PostgreSQL, Redis and Celery workers.

• Automated test suites for the operations and chat modules.